BERT weights. To also enjoy BERT's representations at decoding time, we tie the weights of the decoder and the encoder. Because the Transformer decoder has *source attention* weights (weights for attending to the encoder representations at decoding time) that are not present in BERT, we tie these source-attention weights to the self-attention weights of the encoder (which are tied to the self-attention weights of the decoder). This fully initializes the Transformer model with BERT weights.

Since the encoder and decoder weights are tied, we make them learn distinct representations by adding a FFN $\mathbf{FF_{enc}}$ that transforms the encoder contextualized representations $\mathbf{L_{enc}}$ as

$$\mathbf{H_{enc}} = \texttt{layer-norm}(\texttt{gelu}(W \cdot \mathbf{L_{enc}})),$$

where W is a parameter matrix (Hendrycks and Gimpel, 2016; Ba et al., 2016). Analogously, we add $\mathbf{FF_{dec}}$ to the decoder. To further distinguish the encoder and decoder, we use distinct start and end tokens for input and output sequences. Given m answer tokens $\mathbf{a} = (a_1, \ldots, a_m)$, we form an output sequence with $m+2$ tokens: $\langle$[SOS] $\mathbf{a}$ [EOS]$\rangle$. The output tokens are passed through the decoder and $\mathbf{FF_{dec}}$ to obtain $\mathbf{H_{dec}}$.

Finally, the probability of an answer is defined in the usual manner: Let $\langle \mathbf{a} \rangle = (a_0 \cdots a_{m+1})$ be the output sequence. The decoder outputs the probability $p_{\text{dec}}(a_{i+1} \mid a_0, ..a_i, \mathbf{c}, \mathbf{q})$, and the probability of an answer is:

$$p_{\text{dec}}(\langle \mathbf{a} \rangle \mid \mathbf{c}, \mathbf{q}) = \prod_{i=0}^{m} p_{\text{dec}}(a_{i+1} \mid a_0, ..a_i, \mathbf{c}, \mathbf{q}).$$

As we have a generative model, we can remove the specialized *count* and *arithmetic* heads from §2. Thus, the type head $\mathbf{FF_{typ}}(\mathbf{H_{enc}})$ outputs a distribution $(p_{\mathbf{q}}, p_{\mathbf{c}}, p_{\mathbf{dec}})$ over the context span, question span, and decoder heads.

To improve pre-training on the numeric data (§4), we make two additional modifications.

Digit Tokenization (DT) Conventional wordpiece tokenization treats numbers no differently than any other token. However, computing the value of numbers should be simpler when using digits directly (Wallace et al., 2019). Hence, we tokenize numbers digit-by-digit. For example, a wordpiece $\#\#d_1 \cdots d_k$ where $d_i \in$ {0, . . . , 9} is further split into $\#\#d_1$, ..., $\#\#d_k$. We show in §5.1 that this substantially improves sample complexity when training to perform numerical operations.

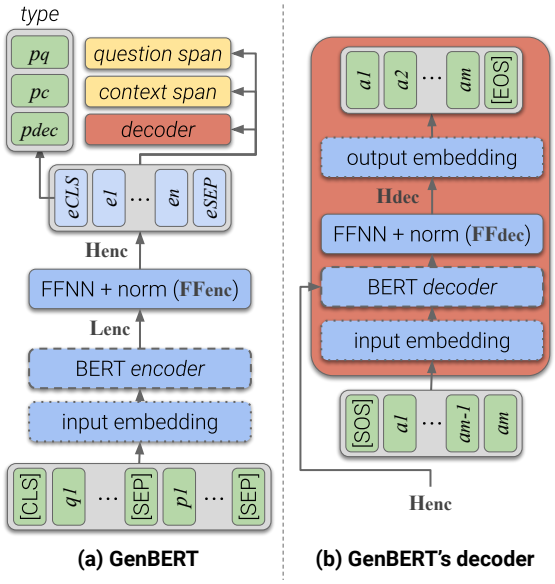

Figure 2: GENBERT's network architecture: (a) a high-level overview of the network, including a generative head (red), two span-extraction heads (yellow), and an answer type head. (b) a closer overview of GENBERT's generative head.

Random Shift (RS) The original Transformer uses absolute positional embeddings for each token. However, in §4, we train on short inputs such as *"1086.1 - 2.54 + 343.8"*. Thus, the model can potentially over-fit and learn to perform numerical reasoning only when numbers are at the beginning of an input. To prevent this, when the input length $n_1 + n_2 + 3 < 512$, we shift all position IDs by a random integer in $(0, 1, \ldots, 512 - (n_1 + n_2 + 3))$.

Training For each span (i, j), a span extraction head h outputs its probability $p_h((i, j) \mid \mathbf{c}, \mathbf{q}, h)$ of being the answer. Let S be the set of spans in the input corresponding to the gold answer. The model loss $\mathbf{L}_{\text{model}}$ marginalizes over all ways in which the answer can be predicted:

$$-\log\left(p_{\mathbf{dec}} \cdot p_{\text{dec}}(\langle \mathbf{a} \rangle) + \sum_{\mathbf{h} \in \mathbf{q},\mathbf{c}} p_{\mathbf{h}} \cdot \sum_{(i,j) \in S} p_h(i, j)\right),$$

where conditionals have been dropped for brevity.

To evaluate the ability of GENBERT to perform numerical reasoning, we initialize it with BERT and fine-tune it on DROP. GENBERT obtains 46.1 EM and 49.3 F_1, roughly 20 points lower than prior models. Thus, we conclude that acquiring numerical reasoning skills from DROP data only is difficult. To remedy this, we will automatically generate training data that will endow GENBERT with numerical skills before training it on DROP.

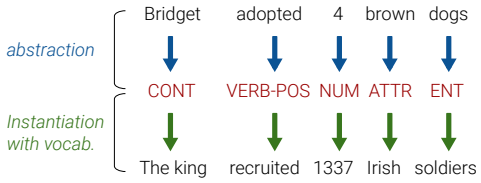

Figure 3: Template extraction and instantiation. A template (in red) is extracted from a MWP sentence, using categories for containers, entities, verbs, attributes and numbers, according to Hosseini et al. (2014). For generation, the categories are instantiated with a domain-specific vocabulary.

4 Pre-training Tasks for Numerical Skills

We now describe two automatically-generated datasets and the multi-task training procedure.

4.1 Generating Numerical Data (ND)

Our first dataset focuses on learning numerical values expressed by tokens and computing numerical operations, i.e., it does not involve textual content. As such, it is easy to craft templates that correspond to various numeric operations. We designed six such templates, described in Table 2. Each template consists of an expression to evaluate and its solution. Further details on their instantiation are provided in §A.1. While the numerical operations were chosen based on DROP, it is trivial to extend them to other domains (Saxton et al., 2019) with different numerical operations.

4.2 Generating Textual Data (TD)

Numeric data is easy to generate, since it does not contain any textual context. However, to tackle NRoT, a model needs to comprehend how numerical operations are expressed in text that refers to events, entities and quantities. This primes us to generate *textual data* from a simple grammar.

While text generation is hard in the general case, we are specifically interested in text that focuses on number manipulations. Therefore, we use the framework of Hosseini et al. (2014), who proposed to model math word problems with a simple structure. In their framework a *world state* consists of *entities*, which are objects that are being counted, and *containers*, which are objects that own entities. Sentences use *verb categories* to describe how the number of entities in a container changes, and thus a world state can be updated given a sentence.

Consider the textual example in Figure 1. the *entities* are soldiers and citizens, and the *containers* are the king and the commander. The verbs (*"had"* and *"received"*) describe the entities the king holds, and how many were passed to the commander.

In this work, we use this framework to automatically generate examples. We *extract templates* that describe changes in the number of entities owned by containers, and automatically *generate question-context pairs* from these templates.

Template extraction To extract templates, we go over sentences from the corpus provided by Hosseini et al. (2014). For each sentence, we use a procedure described by Hosseini et al. (2014) to abstract its tokens to the following categories: numbers (`NUM`), entities (`ENT`), containers (`CONT`) and attributes (`ATTR`). In addition, verbs are abstracted to six categories, each corresponding to a different change in the number of entities owned by containers. Thus, each template fully specifies how to update a *world state*, i.e., the number of entities each container owns. The top part of Figure 3 illustrates the abstraction process. Finally, we count for each extracted template its frequency in the data, and use the top-12 templates for passage generation. Details on the abstraction process, categories used, and extracted templates are in §A.2.

Passage generation Using the extracted templates, we can generate sentences and maintain a world state of all containers and the number of entities they own. We construct a small vocabulary (<100 words) that maps categories to domain-specific words, and use the following procedure to generate passages.

We sample 3-6 templates with replacement, and instantiate them one-by-one (the bottom part of Figure 3 illustrates instantiation). Each template is instantiated by uniformly sampling values from the vocabulary with probability $1 - p$ and from previously generated sentences with probability p. To avoid a collection of unrelated sentences, we set the probability of using previously used values to $p = 0.7$. An example passage is shown in Table 3.

Question generation After generating a passage, the world state holds information about all containers in the passage and the number of entities they hold. In Table 3, the state will include the number of families and rebels of different nationalities in each container (the commander, the householder, and the countries). Based on this world state, numerical reasoning questions can be asked.

To create questions, we craft 13 question templates that are instantiated with objects from the world state. The questions teach the model to track events and perform numeric and discrete operations.

Operation	Template	Example instantiation
signed float combination	$s_1\ f_1\ s_2\ f_2\ s_3\ f_3\ s_4\ f_4$	517.4 - 17484 - 10071.75 + 1013.21
min/max/avg	$o(f_1,\ f_2,\ f_3,\ f_4)$	largest(13.42, 115.5, 72.76)
arg max, arg min	$arg(w_1\ f_1,\ w_2\ f_2,\ w_3\ f_3,\ w_4\ f_4)$	arg min(highish 137.1, sightliness 43.2)
date min/max	$dsup(d_1,\ d_2,\ d_3,\ d_4)$	oldest(June 04, 959; 01 May 959)
date difference	diff in $prd(d_1, d_2)$	diff in days(05 April 112; June 01, 112)
percentage	$pcent\ w\ ::\ w_1\ p_1\%,\ w_2\ p_2\%,\ w_3\ p_3\%,\ w_4\ p_4\%$	percent not sunbird :: sunbird 33.2%, defector 60.77%, molehill 6.03%

Table 2: Templates for generating synthetic numerical examples and the numerical operations required to answer them. **Domains** (defined in App. A.1): $s_i \in \{-, +\}$, $f_i \in \mathbb{R}^+$, $o \in \mathcal{O}$: superlative words like *"longest"*, $arg \in \{\arg\min, \arg\max\}$, $w_i \in \mathcal{W}$: words from NTLK Words Corpus, $d_i \in \mathcal{D}$: dates until Sep 2019, $dsup \in \mathcal{DSUP}$: superlative words like *"latest"*, $prd \in \{$*"days"*, *"months"*, *"years"*$\}$, $p_i \in (0, 100)$, $pcent \in \{$*"percent"*, *"percent not"*$\}$.

P: The commander recruited 1949 Polish families in Spain. The householder recruited 1996 Japanese families in Spain. There were 10913 white rebels and 77 Chinese families in Spain. 6641 British soldiers, 476 asian rebels, and 338 Germans families were recruited in Russia.

Q: How many Japanese families were in Spain?
A: 1996
Q: How many more Japanese families were in Spain than Polish families?
A: 47 (1996-1949)
Q: How many families of Spain were not Polish families?
A: 2073 (4022-1949)

Table 3: An example synthetic passage (P) and questions. Questions (Q) were generated from templates and answers (A) were calculated based on the world state.

Examples for generated questions are shown in Table 3, where answers are computed from the world state. Overall, we create 13 question templates for 7 different "skills", provided in §A.2.

4.3 Training GENBERT on Synthetic Data

For pre-training on ND, we generated 1M examples for training and 10K for validation. For TD, we generated 2.5M examples for training and 10K for validation. For both synthetic datasets, we used the GENBERT model loss, $\mathbf{L}_{\text{model}}$, from §3. To ensure that the model does not lose its language understanding abilities, we employ a multi-task setup, and include a standard *masked LM* objective from BERT. Specifically, given a masked token sequence $\langle \mathbf{m} \rangle$, we compute the contextualized representations, $\mathbf{L}_{\mathbf{enc}}$ and pass them through a feed-forward network $\mathbf{FF}_{\text{mlm}}$. For each masked index i, it outputs the probability $p(a_i \mid i, \langle \mathbf{m} \rangle)$ of the original token a_i. The MLM loss is computed as

$$\mathbf{L}_{\text{mlm}}(\langle \mathbf{m} \rangle) = \text{mean}_{i \in \text{masked}} -\log(p(a_i \mid i, \langle \mathbf{m} \rangle)).$$

Details about the MLM data are in §A.3.

During training, we sample mini-batches from the respective datasets, and minimize the weighted sum of the losses. Concretely, while pre-training on ND and TD, we sample mini-batches X_{ND}, X_{TD} and X_{MLM} and optimize the objective

$$\mathbf{L}_{\text{model}}(X_{\text{ND}}) + \mathbf{L}_{\text{model}}(X_{\text{TD}}) + \lambda \cdot \mathbf{L}_{\text{mlm}}(X_{\text{MLM}}).$$

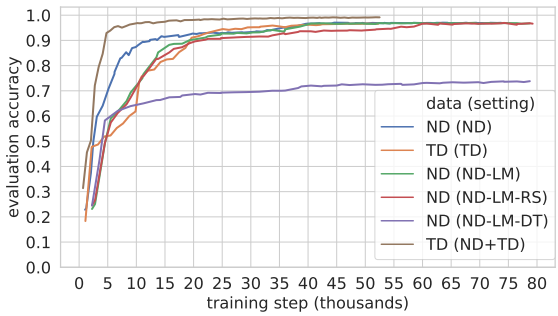

Figure 4: Progression of eval accuracy (EM) of GENBERT, for different pre-training settings listed in §5.1.

5 Experimental Evaluation

We now evaluate our two pre-training steps and their applicability for numerical reasoning tasks. We consider the following variants, aiming to investigate the contributions of ND and TD, the importance of MLM loss, and techniques like DT and RS. In all cases, we initialize GENBERT with BERT-base-uncased, use DT and RS, and include the MLM loss, *except where noted*:

- GENBERT$_{+\text{ND}}$: trained on numerical data.
- GENBERT$_{+\text{ND-LM}}$: trained on ND without the additional MLM loss.
- GENBERT$_{+\text{ND-LM-DT}}$: trained on ND using wordpiece tokenization, without the MLM loss.
- GENBERT$_{+\text{ND-LM-RS}}$: trained on ND without MLM loss and random shift (RS).
- GENBERT$_{+\text{TD}}$: trained on textual data (TD).
- GENBERT$_{+\text{ND+TD}}$: GENBERT$_{+\text{ND}}$ trained on both ND and TD.

5.1 Pre-training Performance

We first ask whether the pre-training procedure allows GENBERT to absorb the intended numerical skills. We observe that across various settings (ND, TD, ND+TD), GENBERT consistently achieves more than 96% accuracy in predicting the correct solution for both ND and TD. Thus, we conclude that indeed a pre-trained LM can learn the designed skills from generated data.